

Git

Francisco Ganacim
ganacim@impa.br

What is it? What is it for?

- Version control
- Collaboration
- Programming tool

Install and configure

- `sudo apt install git`
- `git config --global user.name "Francisco Ganacim"`
- `git config --global user.email "ganacim@impa.br"`
- `git config --global core.editor vim`
- `git config -l`

Working locally

Creating a project

- `mkdir fib`
- `cd fib`
- `git init`
- `git branch -m main`
- `git branch -a`
- `git status`

Configuring the python project

- `python3 -m venv env`
- `vi .gitignore`
- `vi Readme.md`
- `git add .gitignore Readme.md`
- `git commit -m "Project start."`
- `git status`
- `git log`

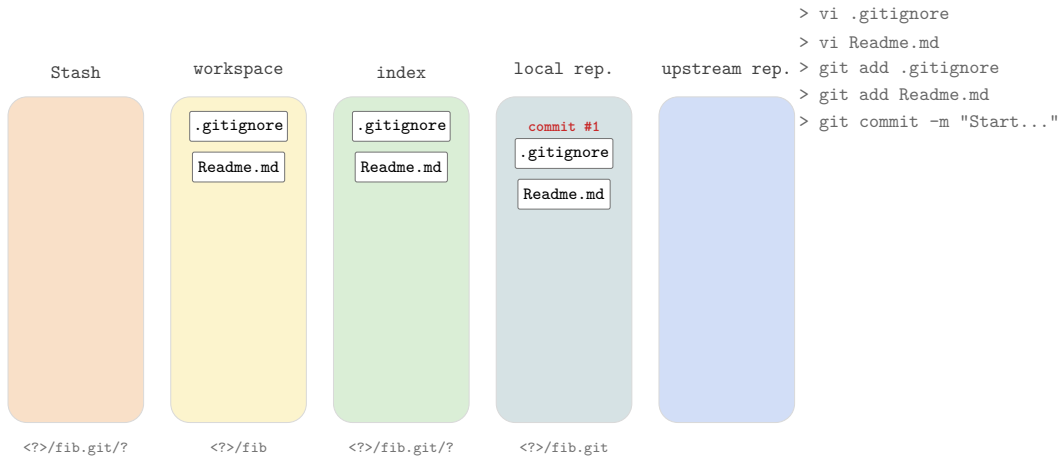
add /env

add some text

.gitignore patterns

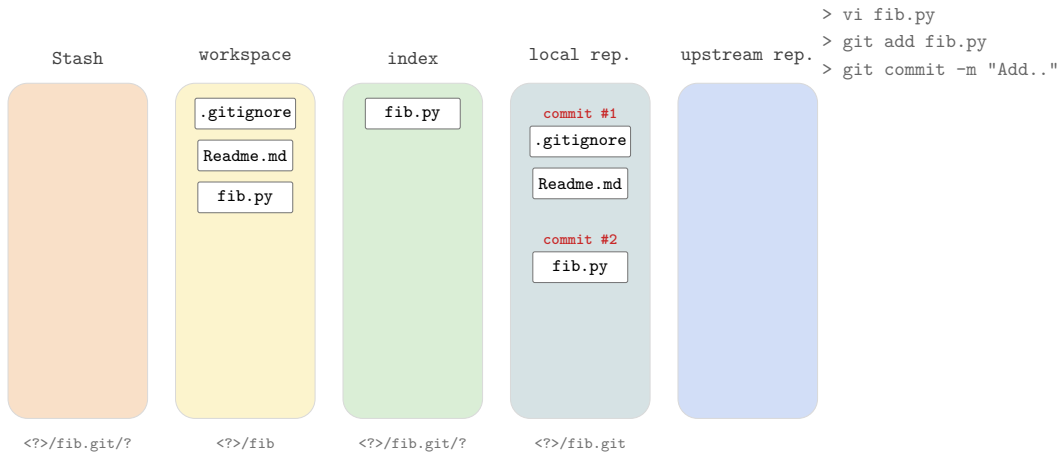
Each line is a pattern; matching files stay untracked.

<code>env/</code>	<i>ignore a directory (trailing /)</i>
<code>*.pyc</code>	<i>glob: any file ending in .pyc</i>
<code>__pycache__/</code>	<i>match these dirs at any depth</i>
<code>/build</code>	<i>anchor to the repo root only</i>
<code>!debug.log</code>	<i>negate: keep this despite *.log</i>
<code># note</code>	<i>full-line comments start with #</i>




```
> vi fib.py
> # edit the file
> source env/bin/activate
> python fib.py
> git status

> git add fib.py
> git commit -m "Add fib.py"
> git status
```



```
> git log --graph --name-status
```

```
* commit 44d6b3e77e31e9678a2a818532417f88abc6de79 (HEAD -> main)
| Author: Francisco Ganacim <ganacim@impa.br>
| Date:   Mon Oct 28 11:06:42 2024 -0300
|
|     Add fib.py
|
|  A   fib.py
* commit 5bbdd907f48d4194f6586b314218a3c5ca870d1e
  Author: Francisco Ganacim <ganacim@impa.br>
  Date:   Mon Oct 28 09:55:51 2024 -0300

      Project start.

  A   .gitignore
  A   README.md
(END)
```

Under the hood: .git

```
> ls .git
HEAD    config  objects/  refs/

> cat .git/HEAD
ref: refs/heads/main                # HEAD points at the current branch

> cat .git/refs/heads/main
5bbdd90...                          # a branch is just a file with a hash
> git cat-file -p HEAD              # a commit = snapshot + parent + author
tree    a1b2c3d...
author Francisco Ganacim <ganacim@impa.br>
```

A branch is a movable pointer — a text file with a 40-char hash. **A commit stores the full tree**, not just a diff.

Undoing changes

```
> git diff                # changes not yet staged
> git diff --staged       # changes staged for commit

> git restore fib.py      # discard edits in workspace
> git restore --staged fib.py # unstage, keep the edits

> git reset --soft HEAD~1 # undo last commit, keep changes
> git revert <hash>       # new commit that undoes another
```

Recovering lost work

```
> git reset --hard HEAD~3      # oops -- did I just lose 3 commits?
> git log --oneline            # ...they're gone from here

> git reflog                  # but every move of HEAD is recorded
9f8e7d6 HEAD@{0}: reset: moving to HEAD~3
a1b2c3d HEAD@{1}: commit: Add fib.py

> git reset --hard HEAD@{1}    # back to safety -- nothing was lost
```

Working with remotes

Remote

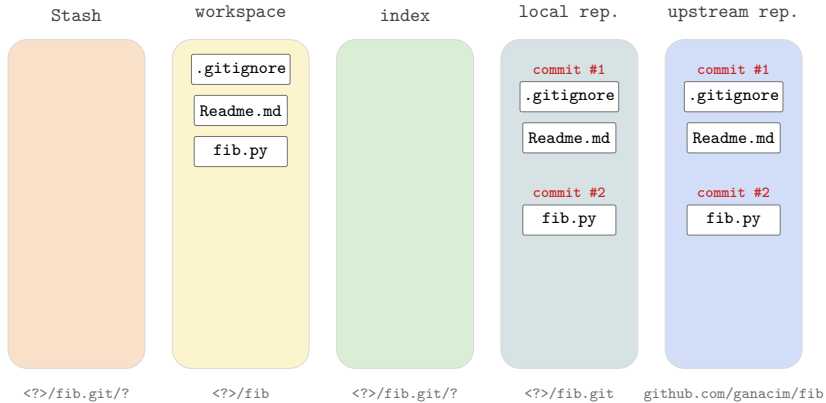
```
> # create an ssh key
> cd ~/.ssh
> ssh-keygen -t ed25519 -C "you@example.com" -f github.ed
> vi config
```

```
Host github.com
    HostName github.com
    User ganacim
    IdentityFile ~/.ssh/github.ed
```


Remote

```
> git remote add origin git@github.com:ganacim/fib.git  
> git branch -M main  
> git push -u origin main
```

> git push -u origin main



Remote (default upstream)

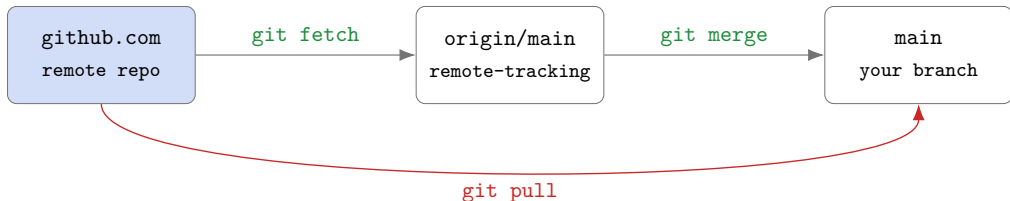
```
> vi fib.py  
> # edit the file  
> git commit -a -m "Format program output"  
> git push
```

Collaboration

```
> git clone git@github.com:ganacim/fib.git
> cd fib

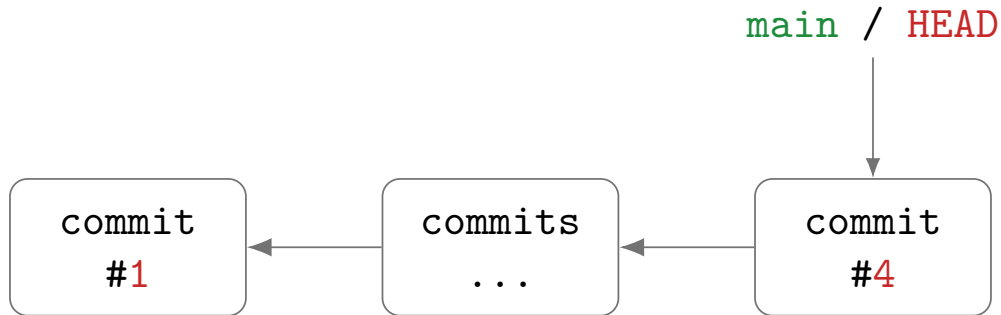
> git fetch          # download remote changes, no merge
> git pull           # fetch + merge into current branch
> git pull --rebase   # fetch + rebase (keep history linear)
```

Fetch, merge, pull



Branching, rebase & merge

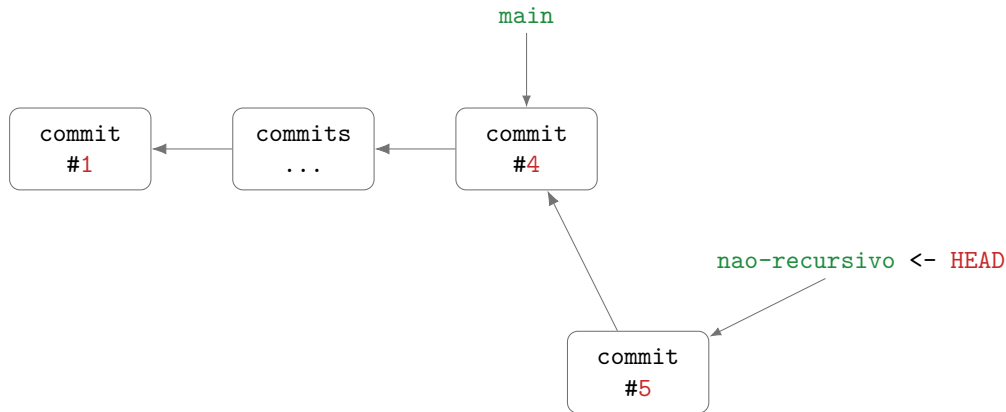
Basic Rebase and Merge



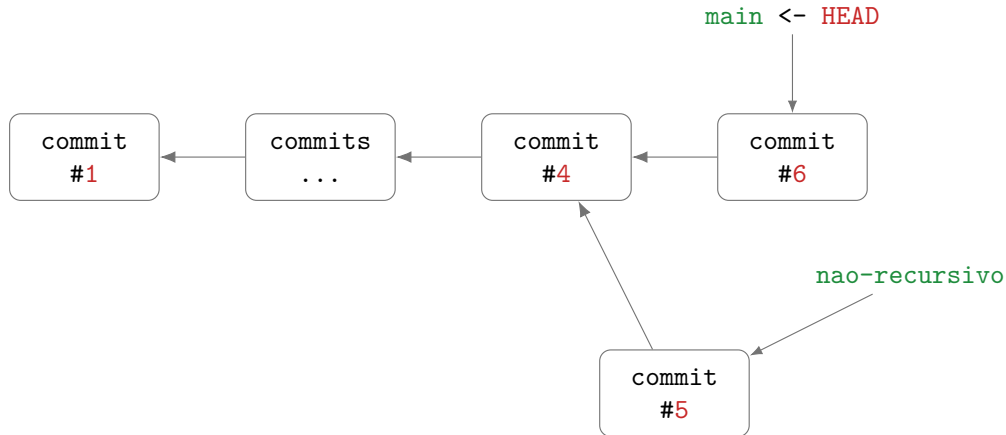
Branching

```
> git switch -c nao-recursive # create + switch to branch  
> git branch -a  
> vi fib.py  
> # edit the file  
> git commit -a -m "Non-recursive version"
```

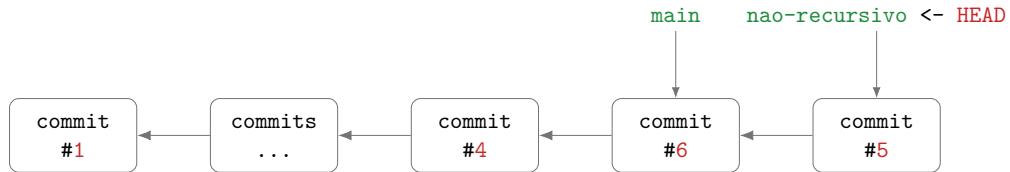

Basic Rebase and Merge



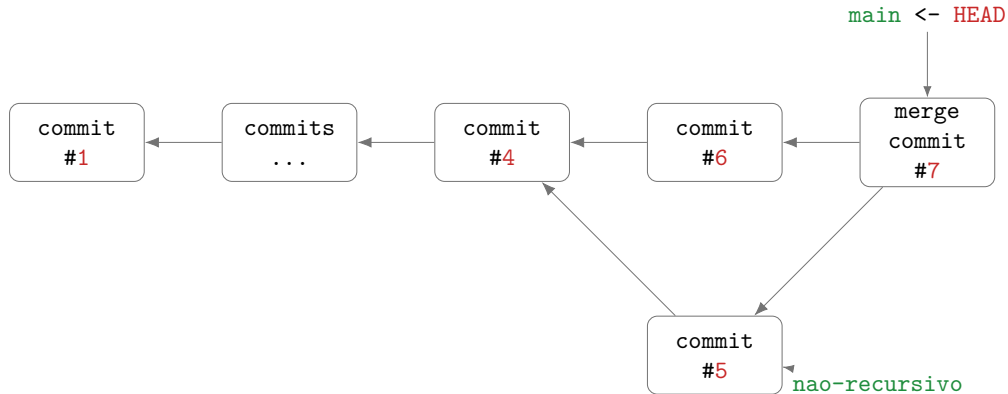
Basic Rebase and Merge



Basic Rebase



Basic Rebase and Merge



Merge conflict

```
> git switch main
> git merge nao-recursive
Auto-merging fib.py
CONFLICT (content): Merge conflict in fib.py
Automatic merge failed; fix conflicts and commit.

> git status
  both modified:   fib.py
```

Resolving a conflict

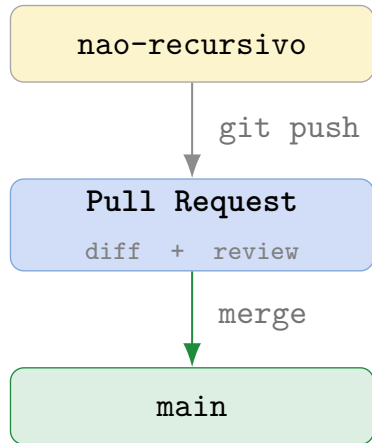
```
<<<<<< HEAD (your branch, main)
def fib(n):
    return fib(n-1) + fib(n-2)
=====
def fib(n):
    a, b = 0, 1
    for _ in range(n): a, b = b, a + b
    return a
>>>>>> nao-recursivo (the branch you merged)
```

```
> # edit fib.py: keep one version, delete the markers
> git add fib.py
> git commit # completes the merge
```

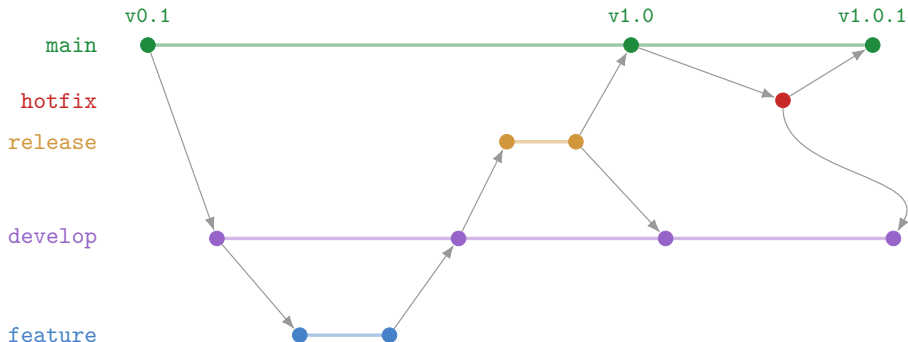
Pull requests

1. Push your branch to GitHub:
`git push -u origin nao-recursive`
2. Open a **Pull Request** on GitHub
3. Teammates review the diff and comment
4. Maintainer merges when approved

A PR lives on GitHub, not inside git —
it is the *conversation* around your branch.



A team workflow: Gitflow



features merge into develop; a release freezes it; main holds tagged releases; hotfix patches production.

after V. Driessen, "A successful Git branching model" —
nvie.com/posts/a-successful-git-branching-model/

Cheat sheet

Everyday

<code>git status</code>	<i>show working state</i>
<code>git add <file></code>	<i>stage changes</i>
<code>git commit -m ".."</code>	<i>save a snapshot</i>
<code>git log</code>	<i>view history</i>
<code>git diff</code>	<i>what changed</i>

Branches & remotes

<code>git switch -c </code>	<i>create a branch</i>
<code>git switch </code>	<i>change branch</i>
<code>git merge </code>	<i>combine branches</i>
<code>git pull</code>	<i>fetch + merge</i>
<code>git push</code>	<i>upload commits</i>



Avaliação do Workshop :)